14.InstructGPT

20 March 2026 18:08

Bigger models don't necessarily mean they follow user intent

alignment not guaranteed with model size

↳ They show how it can be done through fine tuning with human feedback

labeler → prompts → fine-tune using supervised learning → rankings of model outputs → further fine tune using RLHF

↓

improvements in truthfulness & reductions in toxic output generation while having minimal performance regression on public NLP datasets

PPO → used to train the policy (language model)

mechanistically

probability distribution over tokens

$\pi_\theta(y|x)$ a parametrized distribution

Core pipeline SFT → RM → PPO

Iterative loop

Human preferences drive learning

High level motivation

Scaling ≠ Alignment

Alignment = match human intent

Reward model idea

Trained with supervised learning (Ranking loss)

Takes (prompt, response) → scalar

Trained via comparisons

Pre training optimizes → next token likelihood

Humans care about → helpfulness/preference

} different objective spaces

RM gradients come from pairwise comparisons

Policy gradients come from RM scores

} two stage gradient flow system

Reward Model learns

$r_\phi(x, y)$

A latent function over outputs conditioned on prompts

} Induces an order over responses

Not absolute truth

↳ relative preferences ranking

They do not optimize reward directly

They optimize reward from RM

+ penalty for deviating from SFT model

} without this

model exploits reward model

generates unnatural/degenerate outputs

After RLHF → model shifts distribution toward human preferred outputs

But public benchmarks ≠ human preferred distr"

so performance can drop

Pretraining → broad capability space

RLHF → reweights regions of that space

As model size ↑

prior knowledge ↑

RLHF becomes behavior shaping

They say

RM overfits quickly

PPO can exploit RM

} core instability

if no RM → no alignment signal

if no PPO → cannot optimize non differentiable human preference

if no KL penalty → collapse / reward hacking

Without KL

policy will

→ move far from pretrained distribution

Produce

→ unnatural text

→ adversarial outputs that exploit RM

if we directly optimize $\max_y r_\phi(x, y)$

it overoptimizes imperfections in the reward model

Not true human preferences → reward hacking

RM is frozen during PPO

Gradients flow $r_\phi(x, y) \to \pi_\theta$

Policy is updated using RM as a fixed critic

$$\text{Objective}(\phi) = E_{(x,y)\sim D_\pi}\left[r_\phi(x,y) - \beta \log \frac{\pi_\phi^{RL}(y|x)}{\pi^{SFT}(y|x)}\right]$$

reward

↳ Score from reward model

How good humans think something is

KL penalty written as log ratio

how much new policy deviates from SFT

β controls tradeoff

↑β → stay close to SFT

↓β → chase reward more

SFT → defines a manifold of human responses

PPO → moves within that manifold

not arbitrarily outside it

→ This is equivalent to constrained optimization

$\max r_\phi$ such that $D_{KL}(\pi \| \pi_{SFT}) \le \varepsilon$

But implemented as soft penalty instead of hard constraint

Maximize Reward while staying near a safe prior

RM → provides scalar signal

SFT → provides prior distribution

PPO → balances both

if we remove KL

gradients pushed toward regions where RM gives high score

But RM is

imperfect, trained on limited comparisons

Result

Policy finds adversarial points where

RM score is high

humans would not agree

} Systematic Exploitation of Reward Model Errors

Why SFT first?

policy starts random → RL unstable

Why RM?

human preference not differentiable

Why PPO + KL?

RM is imperfect → needs constraint

The above happens if β = 0

β > 0 → policy is forced to stay near π_{SFT}

optimization happens locally, not

β → 0

→ pure reward maximization

→ reward hacking

→ distribution drift

β > 0

constrained optimization

→ stable, human like outputs

without KL → jump to RM peaks anywhere

with KL → trust region around SFT

We need to optimize over $y \sim \pi_\theta(y|x)$ (discrete sequences (tokens)) sampling here is non differentiable →

RL rewrites the gradient as

$$\nabla_\theta \mathbb{E}_{y\sim\pi_\theta}[r_\phi(x,y)] = \mathbb{E}\left[r_\phi(x,y)\,\nabla_\theta \log \pi_\theta(y|x)\right]$$

Instead of differentiating through outputs → we adjust probabilities through outputs

this breaks standard training

because you can't do $\nabla_\theta r_\phi(x,y)$

because y is sampled

no gradient flows through sampling step

SFT → target tokens given

direct gradient

RLHF → no ground-truth

only scalar reward

must use policy gradient

Here objective combines reward + KL

optimization difficulty comes from sampling

↓

RL needed because objective depends on sampled outputs, which are non differentiable

So we use policy gradient to update distribution

we are updating $\nabla_\theta J = r_\phi(x,y)\,\nabla_\theta \log \pi_\theta(y|x)$ → sample a seq y

it has current probability $\pi_\theta(y|x)$

Compute reward $r_\phi(x,y)$

Apply gradient $\nabla_\theta \log \pi_\theta(y|x)$

↓

implies changing the parameters to increase probability of the sequence

if reward is high → strong push

if reward is low → weak/negative push

→ PPO constraint → local stability

How much policy changes per update step

prevents large jumps

Unstable gradients

training collapse

Compares π_{new} vs π_{old}

(Don't move too fast)

↓

only this → small updates

but over many steps policy drifts far from SFT

Slow but inevitable misalignment

→ KL penalty to SFT

How far the policy drifts from π_{SFT}

prevents reward hacking

unnatural language

leaving human like distribution

(Don't move too far)

↳ only this → stays near SFT

large updates & unstable

training instability

As RL progresses $\pi_{SFT} \to \pi_\theta$

RL is trained on old policy outputs but used on new policy outputs

this creates potential mismatch

need for KL constraint

SFT learns $p(y|x) \approx$ human written preferences

It is a behavioral imitation step

RM learns $r_\theta(x|y)$ → preference evaluator, not a generator

RL actually shifts the policy towards higher reward regions

Control comes from KL penalty

PPO constraint

Pipeline (Again !!)

Stage 1 SFT ($\neq$ alignment, ↓ only gives reasonable output)

Objective $\max \log \pi_\theta(y_{human}|x)$

Role puts model in human like response manifold

Stage 2 Reward Model

Input (prompt + multiple responses)

Learns relative preference function

Role converts human judgement → scalar signal

Stage 3 RL (PPO + KL)

Objective maximize reward model output

stay close to SFT

Role Optimize behavior beyond imitation

Overall

Two models exist

(A) policy model (language model)

Starts as pretrained GPT

→ becomes SFT model

→ then updated via RL

(B) Reward Model

Separate network

Trained once

Results

→ Human preference win rate ↑

→ Instruction following ↑

→ Truthfulness ↑ (hallucinations exist)

→ Toxicity ↓

Bias still there

Hallucinations still exist

Over verbosity exists

smaller RLHF model > GPT3

Alignment > scale

Improvements on API style prompts > traditional NLP benchmarks

Performance regression on NLP datasets

RLHF changes behavior

Not knowledge

Limitations → imperfect reward model

→ Alignment is shallow (hallucinations, bias, overconfidence)

→ Uncertain generalization

Distb^n mismatch (grows during training)

RM trained on $y \sim \pi_{SFT}$

RL uses $y \sim \pi_\theta$